

# Project TODOs

---

## Security

- ☒ **1. Message length cap alignment** — bump `max_length` in `backend/models/chat.py` from 200 → 4000; add matching `max(4000)` to `ChatRequestSchema` in `frontend/src/lib/schemas.ts`
- ☒ **2. Rate limiting** — add `slowapi`, apply per-IP limit on `/chat` (e.g. 20 req/min), add JSON 429 handler, expose `X-RateLimit-*` headers via CORS
- ☒ **3. HTTP security headers** — add `SecurityHeadersMiddleware` to `middleware.py` injecting `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, `Permissions-Policy`
- ☒ **4. API key authentication** — `Authorization: Bearer` dependency on `/chat`, `API_KEY` env var on backend, `VITE_API_KEY` on frontend injected in `api.ts`
- ☐ **5. conversation\_id scoping** — bind conversation IDs to authenticated identity server-side. **UNBLOCKED** (#4 done; `verify_clerk_user` now returns the Clerk user id `payload["sub"]`). Real gap: `conversation_id` comes from the client and isn't bound to the user, so one user could pass another's id and read their `MemorySaver` history. **Part of the user-identity cluster** (with "Per-user `MAX_QUESTIONS`" + #17b) — all need state keyed by Clerk user id in a shared store.
- ☐ **6. XSS / markdown safety** — add `DOMPurify` or safe markdown renderer (*defer until markdown rendering is added*)
- ☐ **7. Dependency vulnerability scan** — run once and fix findings. ⚠️ Verify the command: `uv audit` may not exist; the standard tool is `pip-audit` (`uvx pip-audit`). Intent stands regardless of the exact command.

## Production hardening

- ☐ In-process `MemorySaver` and `question_counts` dict reset on restart and are not shared across workers — replace with `langgraph-checkpoint-postgres` (or Redis-backed checkpointer) and a Redis/DB-backed counter before multi-worker deployment. **Now actively biting:** Render's free-tier spin-down wipes this on every cold start (not just on deploy), so conversation history is already lost in prod. **Synergy:** if the \$19 transcript migration adds Postgres, `langgraph-checkpoint-postgres` can share that DB. (This is the single home for the old RAG #16b decision.)

---

## RAG Pipeline Build

Full architecture and design decisions are documented in `rag_plan.md`.

### Setup

- ☐ **1. Resolve open decisions** — see `rag_plan.md` §16:
  - ☒ **16a. Auth** — RESOLVED: switched to Clerk (deployed, working).
  - ☐ **16b. Conversation history** — DUPLICATE of "Production hardening / MemorySaver" above; tracked there. (Kept `MemorySaver` for now; see that item for the persistence plan.)
  - ☐ **16c. Transcript quality** — DEMOTED to backlog: the pipeline shipped without a pre-ingest cleaning pass and answers are grounded/working, so this is no longer a build blocker. Revisit only if caption errors degrade answer quality.

- ☒ **2. Install Ollama** — download from ollama.com, run `ollama pull llama3.1:8b`, verify with `ollama run llama3.1:8b`
- ☒ **3. Add Python dependencies** — `uv add openai pinecone tiktoken langchain-ollama langchain_openai`
- ☒ **4. Create Pinecone index** — in Pinecone console: serverless, us-east-1, cosine, dimension 1536, name `guitar-rag`
- ☒ **5. Add new env vars to .env** — `OPENAI_API_KEY`, `PINECONE_API_KEY`, `PINECONE_INDEX_NAME`, `EMBEDDING_PROVIDER`, `LLM_PROVIDER`, `OLLAMA_MODEL`, `OLLAMA_BASE_URL`, `CLAUDE_MODEL`

## Ingestion pipeline (one-time batch — no async needed)

- ☒ **6. Create backend/ingestion/ folder** — add `__init__.py` (empty)
- ☒ **7. Write backend/ingestion/chunker.py** — `filter_transcript` (noise removal) + `chunk_transcript` (~400-token chunks, per-chunk `start_time`, metadata). Non-overlapping done; ~50-token overlap still TODO.
- ☒ **8. Test chunker.py** — `test_chunker.py` with 4 passing tests covering filter (drop/scrub) and chunk (single-chunk flush + forced split)
- ☒ **8c. Add ~50-token overlap to chunk\_transcript** — carry the last few segments of each chunk into the start of the next so context isn't lost at chunk boundaries (see rag\_plan.md §4). `OVERLAP_TOKEN_SIZE=50` constant; seam test asserts chunk N's tail reappears at chunk N+1's head.
- ☒ **8b. Add Transcript/Segment Pydantic models (validate at the boundary)** — the scraper (YouTube API) and JSON load are the real trust boundaries, not the internal `Chunk` (which stays a dataclass).
  - ☒ Define `Segment` + `Transcript` Pydantic models in a module both `data_gathering/` and `backend/` can import (or, if keeping them decoupled, just validate at the loader)
  - ☒ Scraper: build + validate `Transcript` before writing, save via `.model_dump_json()` — bad YouTube data fails at the source
  - ☒ Loader: read back via `Transcript.model_validate_json(...)` so the pipeline can trust the data
  - ☒ Flatten `segments` to `list[Segment]` while re-scraping (kills the `data["segments"][0][0]` quirk — cheap to do now with few files; needs a re-scrape)
  - ☒ Change `chunk_transcript(data: dict) → chunk_transcript(transcript: Transcript)` so it uses `transcript.video_id / transcript.segments` (typed, no `KeyError`); update `test_chunker.py` fixture accordingly
- ☒ **9. Write backend/ingestion/embeddings.py** — `EmbeddingProvider` Protocol, `OpenAIEmbeddingProvider` class, `get_embedding_provider()` factory
- ☒ **10. Test embeddings.py** — embed a handful of chunks, print vector length (should be 1536)
- ☒ **11. Write backend/ingestion/ingest.py** — walk all transcript JSONs → chunk → embed in batches of 100 → upsert to Pinecone. Vector store extracted to top-level `vector_store.py` (`VectorRecord` + `VectorStore` Protocol + `PineconeVectorStore` + factory), shared by ingest & rag.
- ☒ **12. Run ingest.py** — 186 chunks sent, 186 vectors in Pinecone (verified count + metadata shape)

## RAG retrieval (used at request time)

- ☒ **13. Create backend/rag/ folder** — add `__init__.py` (empty)
- ☒ **14. Write backend/rag/retriever.py** — `Retriever` class (providers built once), embed query → `vector_store.query` → assemble context string + list of `Source` objects

- ☒ **15. Test `retriever.py`** — ran via `__main__`, confirmed top-k Sources (title/url/instructor/start\_time) are relevant

## Backend wiring

- ☒ **16. Add Source model + wire into ChatResponse** — `models/source.py` (Pydantic, for API serialization), `ChatResponse.sources: list[Source]`
- ☒ **17. Update `backend/agent.py`** — system prompt replaced with guitar-scoped + RAG-grounding version; `get_llm()` factory extracted to `backend/llm.py` (env-driven `LLM_PROVIDER`, Ollama default with `OLLAMA_BASE_URL/OLLAMA_BASE_MODEL`; reuses langchain `BaseChatModel` as the abstraction)
  - ☐ **17b. Revisit `MAX QUESTIONS`**. (*Premise updated: prod is now Groq, not Ollama.*) It was a Claude-spend cap; prod now runs **free Groq** and dev runs local Ollama, so the *cost* rationale is gone either way. Remaining value is purely **abuse protection** — decide whether to keep it for that or drop it. Keep the slowapi per-IP rate limit regardless (abuse/hardware protection, independent of LLM cost). Note the query path still calls OpenAI `embed_query` per question unless embeddings are localized (forces a full re-ingest — see "Localize embeddings"). **Overlaps with the user-identity cluster** (see Security #5 + "Per-user `MAX QUESTIONS`").
- ☒ **18. Update `backend/services/chat_service.py`** — module-level `Retriever`, retrieve before agent invoke, augmented prompt (context + question), return sources
- ☒ **19. Smoke test via API** — POST `/chat` returned a grounded reply (direct Jack Ruch quotes) + 5 sources + `questions_remaining`. ☒ Full R→A→G loop verified end to end.

## Frontend

- ☒ **20. Update `frontend/src/lib/schemas.ts`** — `SourceSchema` added; `ChatResponseSchema.sources: z.array(SourceSchema)`; `start_time` is `z.number().nullable()`
- ☒ **21. Update `frontend/src/hooks/useChat.ts`** — sources threaded into `Message`; token fetched via `useAuth().getToken()` and passed to `sendMessage`
- ☒ **22. Create `frontend/src/components/chat/sourceCard.tsx`** — clickable card (title, snippet excerpt, timestamp, `CirclePlay` icon, `&t=<sec>s` deep-link)
- ☒ **23. Update `frontend/src/components/chat/MessageBubble.tsx`** — source cards rendered below the assistant answer (outside the scroll box)

## Done

- ☒ **24. End-to-end test** — verified live: logged-in user asks a guitar question, gets a grounded answer + clickable timestamped source cards. Full deployed stack works (Vercel → Render → Groq hosted Llama; originally tested via Cloudflare tunnel → local Ollama, since retired).

## Backlog / nice-to-have

- ☐ **Pattern B — agentic RAG (retrieval-as-a-tool)** — evolve the chat service from Pattern A (always retrieve → inject context → invoke) to exposing `retriever.retrieve` as an agent **tool**, letting the LLM decide when/how often to search. Fits langgraph's tool-calling model. Pros: skip retrieval for non-guitar turns, multi/refined searches. Cons: less predictable, harder to test/guarantee grounding, `sources` must be collected from the tool calls the model actually made. See `rag_plan.md` §10 for the full write-up. Defer until Pattern A works end to end (incl. frontend source cards).

- ☐ **Split `CLAUDE.md` (currently gitignored, local-only)** — if the repo ever wants contributor docs, extract the *neutral* project conventions (import roots, how to run tests, git workflow, secret guardrails) into a committed `CLAUDE.md`, and keep the *personal* mentorship/training framing in a local-only file. Keeps the repo professional without publishing the personal training process.

## Deployment (LIVE)

**Architecture:** Vercel (frontend, static) → Render (FastAPI backend) → **Groq** (hosted `llama-3.1-8b-instant`, free tier). Pinecone / Clerk / OpenAI-embeddings are managed. The deployed app is **fully laptop-independent** — no tunnel, no local process.

**Dev vs prod LLM split** (via the `get_llm()` factory + `LLM_PROVIDER`):

- **Prod (Render):** `LLM_PROVIDER=groq` → hosted Llama, always-on, \$0, fast.
- **Dev (local):** `LLM_PROVIDER` unset → defaults to `ollama (llama3.1:8b)`, free/offline.
- The `anthropic` branch remains a configurable fallback (Claude Haiku).

*History:* prod originally ran local Ollama exposed via a Cloudflare quick tunnel; that required the dev machine + `cloudflared` to be up and minted a new URL every restart. Replaced by Groq (2026-06-14) — tunnel retired, named `ollama_tunnel` left idle/unused.

**Config that bit us (remember these):**

- Render **Start Command:** `uvicorn api:app --host 0.0.0.0 --port $PORT` (NOT 127.0.0.1, NOT hardcoded port). **Build Command:** `pip install -r requirements.txt`. Root Directory = `backend`. (Build installs, Start serves — different fields.)
- `backend/requirements.txt` committed (Render needs a pip-installable list; pyproject is at repo root, app is in backend/).
- Vercel: `VITE_*` are **build-time** (rebuild to change). Set env vars for **all** environments. Production Branch = `main`. Guard `VITE_CLERK_PUBLISHABLE_KEY` or tsc fails.
- **Mirror rule:** `CORS_ORIGINS` (Render) = the **Vercel** URL; `VITE_API_URL` (Vercel) = the **Render** URL. `CORS_ORIGINS` also drives Clerk `authorized_parties`.
- **LLM env (Render):** `LLM_PROVIDER=groq` + `GROQ_API_KEY` (from console.groq.com). Optional `GROQ_MODEL` (defaults to `llama-3.1-8b-instant`). Set the key *before* the redeploy or `/chat` errors (builds `ChatGroq` with no key). `OLLAMA_BASE_URL` on Render is now dead — `LLM_PROVIDER=groq` never reads it (safe to delete).
- (*Retired — kept for history*) Cloudflare tunnel: `cloudflared tunnel --url http://localhost:11434 --http-host-header localhost` — the `--http-host-header localhost` flag was REQUIRED (Ollama 403s the proxied Host header otherwise), and the quick-tunnel URL changed on every restart (had to update Render `OLLAMA_BASE_URL` each time). Groq removed all of this.

**Deploy follow-ups:**

- ☒ **Render free-tier cold start** — DONE (branch `slow-backend-error-handling`): 90s `AbortSignal.timeout` in `api.ts` → friendly `ApiError(408)`; 5s delayed "waking up" hint via `setTimeout` in `MessageList`. Chose UI handling over a keep-warm pinger (extra moving pieces not worth it for a portfolio). Paid tier (~\$7/mo) removes spin-down if ever needed.

- ☐ **Streaming responses** — UX win (stream tokens so it *feels* fast). Touches agent/chat\_service + frontend.
- ☒ **Faster model / real GPU** — SOLVED by Groq: hosted `llama-3.1-8b-instant` is fast and off the dev machine. (Streaming, above, is still a nice extra UX win.)
- ☒ **Named Cloudflare tunnel** — OBSOLETE: Groq removed the tunnel entirely, so there's no `OLLAMA_BASE_URL` to keep stable in prod.
- ☐ **.env.example** — document all required env vars (names + placeholders, no secrets): `CLERK_SECRET_KEY`, `PINECONE_API_KEY`, `OPENAI_API_KEY`, `CORS_ORIGINS`, `LLM_PROVIDER`, `GROQ_API_KEY`, `GROQ_MODEL`, `OLLAMA_BASE_URL`, `OLLAMA_BASE_MODEL`, `YOUTUBE_API_KEY`.
- ☐ **Per-user MAX\_QUESTIONS** — `verify_clerk_user` returns the user id; key counts by user (needs a shared store, not the in-memory dict, for multi-instance/restart-safe). **Part of the user-identity cluster** (with Security #5 + #17b) — all three need state keyed by Clerk user id in a shared store; tackle together.
- ☐ **Localize embeddings** (`nomic-embed-text`) to drop the OpenAI dependency — requires a full re-indexing (incompatible vectors). Deliberate, separate project.

#### data\_gathering cleanup (in progress):

- ☐ Commit the cleaned scraper files (`youtube_video_id_generator.py` now env-reads `YOUTUBE_API_KEY`; redundant keyword-search scraper deleted; leaked Google key ROTATED). Fix any remaining trailing-comma/tuple issues before committing.
- ☐ Decide: **move data\_gathering to its own repo** (it's seeding/ETL, not the agent). Leaning separate. **Now coupled to the datastore migration** — see `rag_plan.md` §19: splitting the repo breaks the JSON-on-disk contract, so the plan is to replace it with a shared Postgres DB (backend = schema authority, scraper = thin writer, two repos not three). Git split option = **A** (fresh init in new repo + `git rm` here; dead key already rotated, no history rewrite needed).
- ☐ ~~`ingest.py` — `instructor-CLI` arg~~ — **SUPERSEDED by `rag_plan.md` §19**: the Postgres `status` lifecycle (`scraped` → `ingested`) makes ingest incremental by construction (query unprocessed rows), which is the real version of this hack. Build that instead.
- ☐ **Datastore migration (Postgres + SQLAlchemy 2.0 + Alembic)** — design in `rag_plan.md` §19. Resolve the 5 open questions there before building (Mongo-vs-Postgres confirm, where Postgres is hosted, schema sharing, migration ownership, backfill of existing JSON).